

Vector Field - Reactive Algorithms Pt 5

A Stream with a Spring

This one is going to have less visualizations than the first.

Max Flow Finder

In [1]:

```
1  ## 1. Import Libraries
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # 2. Define constants
6  x_start = y_start = 0
7  x_end = y_end = 10
8  quiver_step_size = 0.2
9
10 x_source = 3
11 y_source = 5
12
13 robot_1_start = [8,8]
14 robot_2_offset = [0.3,0]
15 robot_3_offset = [0,0.3]
16
17 robot_step_size = 0.5
18
19
20
21 def spring_stream(x, y, x_source, y_source):
22
23     # Establishing a Uniform Flow
24     u = np.ones_like(x)
25     v = np.zeros_like(x)
26
27
28     # Calculating A Source
29     x_centre = x - x_source
30     y_centre = y - y_source
31     r = np.sqrt(x_centre**2 + y_centre**2)
32     r[r==0] = 1e-6
33
34     # Adding the uniform flow to the spring source
35     u = u + x_centre / r**2
36     v = v + y_centre / r**2
37
38     return u, v
39
40 def robot_readings(x, y, x_source, y_source):
41
42     # Centring over sink
43     x_centre = x - x_source
```

```

44     y_centre = y - y_source
45
46     # Calculate the distance r from the center
47     dist = np.sqrt((x-x_source)**2 + (y-y_source)**2)
48
49     # Establishing a Uniform Flow
50     u = np.ones_like(x)
51     v = np.zeros_like(x)
52
53     # Adding the uniform flow to the spring source
54     u = u + x_centre / dist**2
55     v = v + y_centre / dist**2
56
57     return u, v
58
59 def composite_vector(u,v):
60
61     mag = np.sqrt(u**2 + v**2)
62     angle = np.arctan2(v,u)
63
64     return mag, angle
65
66 def plot_robot_position(pos1,pos2,pos3):
67
68     plt.scatter(pos1[0],pos1[1], marker='o', color='blue')
69     plt.scatter(pos2[0],pos2[1], marker='o', color = 'yellow')
70     plt.scatter(pos3[0],pos3[1], marker='o', color = 'green')
71
72
73 def update_robot_position(pos1,pos2,pos3, slope_y, step_size):
74
75     if np.sign(slope_y)>0:
76         pos1[1] += step_size
77         pos2[1] += step_size
78         pos3[1] += step_size
79     else:
80         pos1[1] -= step_size
81         pos2[1] -= step_size
82         pos3[1] -= step_size
83
84     return pos1, pos2, pos3
85
86 # 4. Initialize

```

```

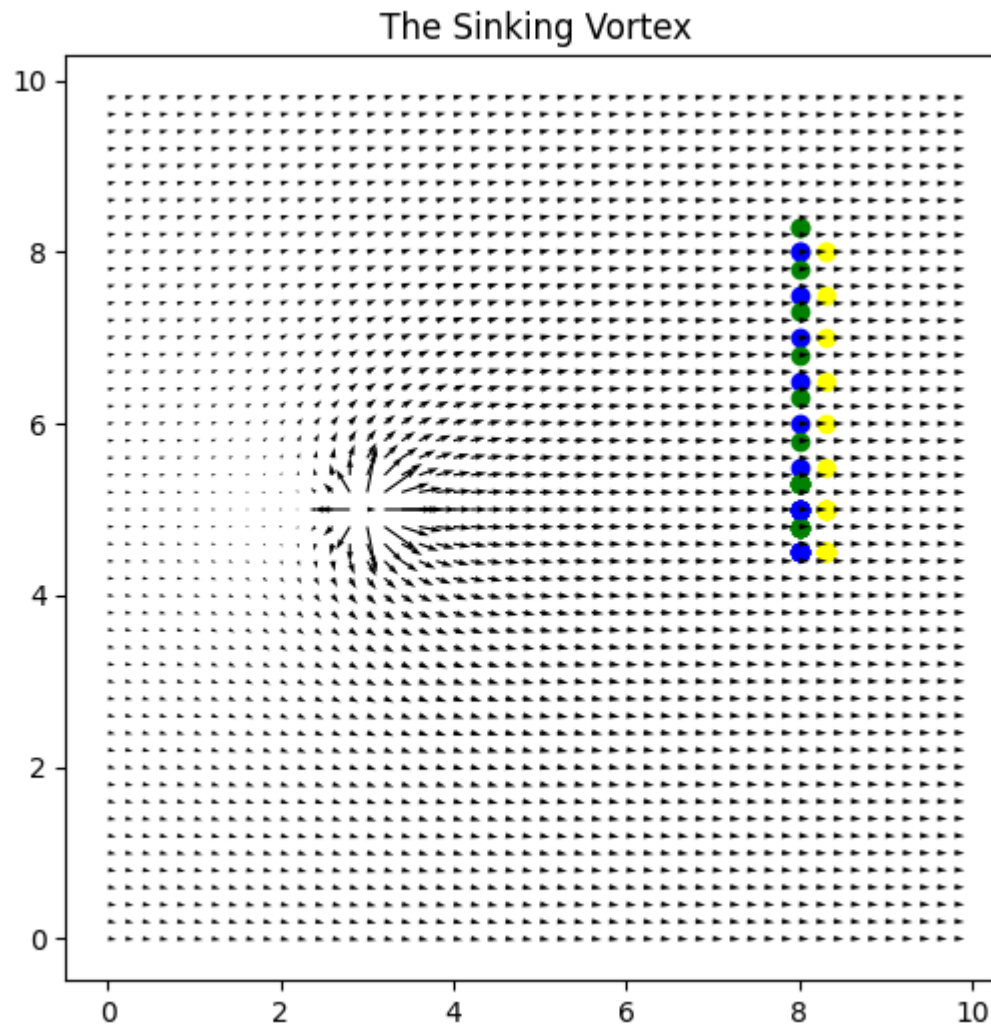
87
88 # Initialize The Sinking Vortex Field
89 x = np.arange(x_start, x_end, quiver_step_size)
90 y = np.arange(y_start, y_end, quiver_step_size)
91 X, Y = np.meshgrid(x, y)
92 u, v = spring_stream(X, Y, x_source, y_source)
93
94 # Initialize the Robot's positions
95 pos1=robot_1_start
96 pos2=[(robot_1_start[0] + robot_2_offset[0]),(robot_1_start[1] + robot_2_offset[1])]
97 pos3=[(robot_1_start[0] + robot_3_offset[0]),(robot_1_start[1] + robot_3_offset[1])]
98
99 # Initialize output Plot
100 plt.figure(figsize=(6, 6))
101 plot_robot_position(pos1,pos2,pos3)
102
103 # 5. Main
104
105 for _ in range(1,20):
106     # Check Break Criteria
107
108     # Find u,v, angle and magnitude at each Robot
109
110     [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
111     [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
112     [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)
113
114     [mag1, angle1] = composite_vector(u1,v1)
115     [mag2, angle2] = composite_vector(u2,v2)
116     [mag3, angle3] = composite_vector(u3,v3)
117
118     # Plot Robot Position
119     plot_robot_position(pos1,pos2,pos3)
120
121     # Update Robot Position
122     #angle = -(angle1 + angle2 + angle3)/3
123     #slope_x = (mag2 - mag1) / robot_2_offset[0]
124     slope_y = (mag3 - mag1) / robot_3_offset[1]
125     #angle = np.arctan2(slope_y,slope_x) #+ np.pi/2
126
127     # Update Robot Position
128
129     [pos1, pos2, pos3] = update_robot_position(pos1,pos2,pos3,slope_y, robot_step_size)

```

```

130
131
132 # 6. Plot Outcome
133
134
135 plt.quiver(x, y, u, v)
136 plt.title(' The Sinking Vortex')
137 #plot_extra_features(x, y, u, v, x_source, y_source)
138 plt.show()
139

```



Source Finder

In [4]:

```
1  ## 1. Import Libraries
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # 2. Define constants
6  x_start = y_start = 0
7  x_end = y_end = 10
8  quiver_step_size = 0.2
9
10 x_source = 3
11 y_source = 5
12
13 robot_1_start = [8,8]
14 robot_2_offset = [0.3,0]
15 robot_3_offset = [0,0.3]
16
17 robot_step_size = 0.5
18
19
20
21 def spring_stream(x, y, x_source, y_source):
22
23     # Establishing a Uniform Flow
24     u = np.ones_like(x)
25     v = np.zeros_like(x)
26
27
28     # Calculating A Source
29     x_centre = x - x_source
30     y_centre = y - y_source
31     r = np.sqrt(x_centre**2 + y_centre**2)
32     r[r==0] = 1e-6
33
34     # Adding the uniform flow to the spring source
35     u = u + x_centre / r**2
36     v = v + y_centre / r**2
37
38     return u, v
39
40 def robot_readings(x, y, x_source, y_source):
41
42     # Centring over sink
43     x_centre = x - x_source
```



```

44     y_centre = y - y_source
45
46     # Calculate the distance r from the center
47     dist = np.sqrt((x-x_source)**2 + (y-y_source)**2)
48
49     # Establishing a Uniform Flow
50     u = np.ones_like(x)
51     v = np.zeros_like(x)
52
53     # Adding the uniform flow to the spring source
54     u = u + x_centre / dist**2
55     v = v + y_centre / dist**2
56
57     return u, v
58
59 def composite_vector(u,v):
60
61     mag = np.sqrt(u**2 + v**2)
62     angle = np.arctan2(v,u)
63
64     return mag, angle
65
66 def plot_robot_position(pos1,pos2,pos3):
67
68     plt.scatter(pos1[0],pos1[1], marker='o', color='blue')
69     plt.scatter(pos2[0],pos2[1], marker='o', color = 'yellow')
70     plt.scatter(pos3[0],pos3[1], marker='o', color = 'green')
71
72
73 def update_robot_position(pos1,pos2,pos3, angle, step_size):
74
75     pos1[0] += step_size*np.cos(angle)
76     pos1[1] += step_size*np.sin(angle)
77     pos2[0] += step_size*np.cos(angle)
78     pos2[1] += step_size*np.sin(angle)
79     pos3[0] += step_size*np.cos(angle)
80     pos3[1] += step_size*np.sin(angle)
81
82     return pos1, pos2, pos3
83
84 # 4. Initialize
85
86 # Initialize The Sinking Vortex Field

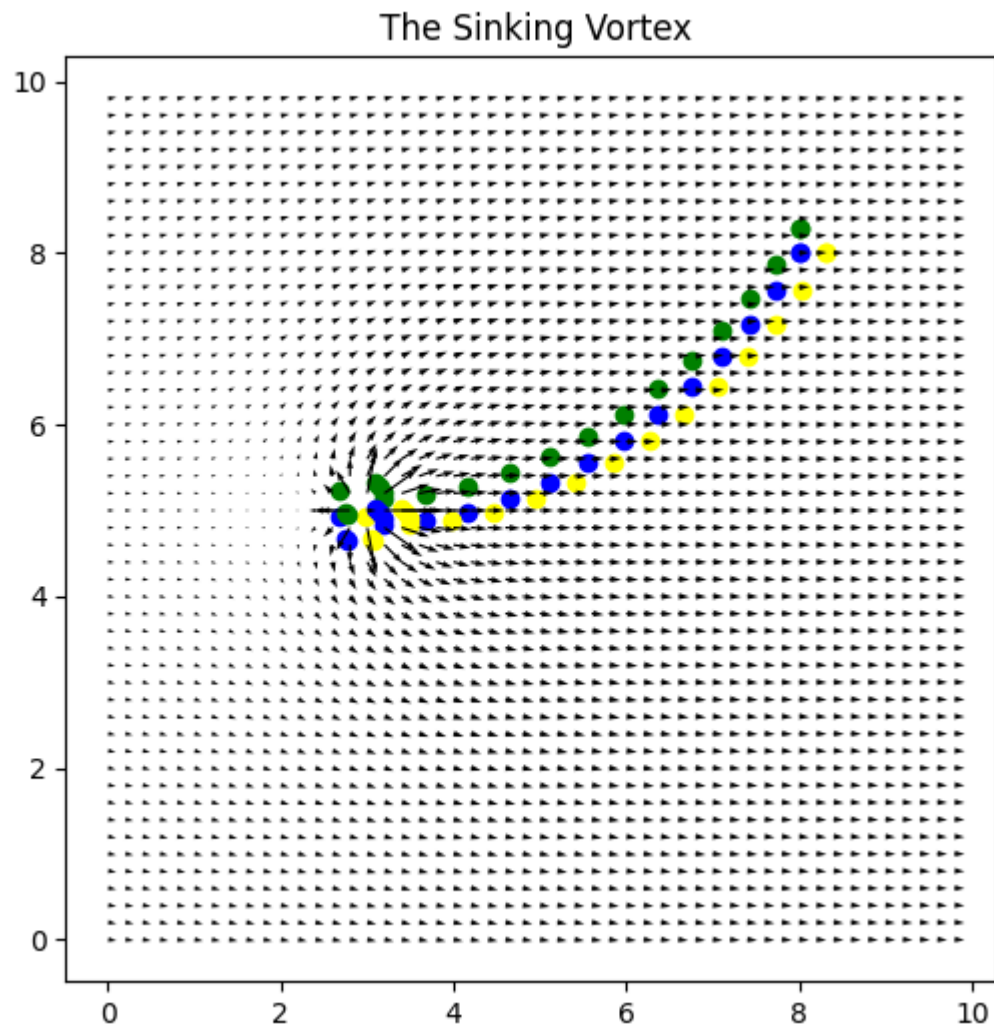
```

```

87 x = np.arange(x_start, x_end, quiver_step_size)
88 y = np.arange(y_start, y_end, quiver_step_size)
89 X, Y = np.meshgrid(x, y)
90 u, v = spring_stream(X, Y, x_source, y_source)
91
92 # Initialize the Robot's positions
93 pos1=robot_1_start
94 pos2=[(robot_1_start[0] + robot_2_offset[0]),(robot_1_start[1] + robot_2_offset[1])]
95 pos3=[(robot_1_start[0] + robot_3_offset[0]),(robot_1_start[1] + robot_3_offset[1])]
96
97 # Initialize output Plot
98 plt.figure(figsize=(6, 6))
99 plot_robot_position(pos1,pos2,pos3)
100
101 # 5. Main
102
103 for _ in range(1,20):
104     # Check Break Criteria
105
106     # Find u,v, angle and magnitude at each Robot
107
108     [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
109     [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
110     [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)
111
112     [mag1, angle1] = composite_vector(u1,v1)
113     [mag2, angle2] = composite_vector(u2,v2)
114     [mag3, angle3] = composite_vector(u3,v3)
115
116     # Plot Robot Position
117     plot_robot_position(pos1,pos2,pos3)
118
119     # Update Robot Position
120     #angle = -(angle1 + angle2 + angle3)/3
121     slope_x = (mag2 - mag1) / robot_2_offset[0]
122     slope_y = (mag3 - mag1) / robot_3_offset[1]
123     angle = np.arctan2(slope_y,slope_x) ## np.pi/2
124
125     # Update Robot Position
126
127     [pos1, pos2, pos3] = update_robot_position(pos1,pos2,pos3,angle, robot_step_size)
128
129

```

```
130 # 6. Plot Outcome
131
132
133 plt.quiver(x, y, u, v)
134 plt.title(' The Sinking Vortex')
135 #plot_extra_features(x, y, u, v, x_source, y_source)
136 plt.show()
137
```



Fastest Flow Finder - 2 steps, straight line

In [2]:

```
1  ## 1. Import Libraries
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # 2. Define constants
6  x_start = y_start = 0
7  x_end = y_end = 10
8  quiver_step_size = 0.2
9
10 x_source = 3
11 y_source = 5
12
13 robot_1_start = [8,8]
14 robot_2_offset = [0.5,0.5]
15 robot_3_offset = [1,1]
16 robot_offset = 0.5
17
18 robot_step_size = 1.1
19
20
21
22 def spring_stream(x, y, x_source, y_source):
23
24     # Establishing a Uniform Flow
25     u = np.ones_like(x)
26     v = np.zeros_like(x)
27
28
29     # Calculating A Source
30     x_centre = x - x_source
31     y_centre = y - y_source
32     r = np.sqrt(x_centre**2 + y_centre**2)
33     r[r==0] = 1e-6
34
35     # Adding the uniform flow to the spring source
36     u = u + x_centre / r**2
37     v = v + y_centre / r**2
38
39     return u, v
40
41 def robot_readings(x, y, x_source, y_source):
42
43     # Centring over sink
```

```

44     x_centre = x - x_source
45     y_centre = y - y_source
46
47     # Calculate the distance r from the center
48     dist = np.sqrt((x-x_source)**2 + (y-y_source)**2)
49
50     # Establishing a Uniform Flow
51     u = np.ones_like(x)
52     v = np.zeros_like(x)
53
54     # Adding the uniform flow to the spring source
55     u = u + x_centre / dist**2
56     v = v + y_centre / dist**2
57
58     return u, v
59
60 def composite_vector(u,v):
61
62     mag = np.sqrt(u**2 + v**2)
63     angle = np.arctan2(v,u)
64
65     return mag, angle
66
67 def plot_robot_position(pos1,pos2,pos3):
68
69     plt.scatter(pos1[0],pos1[1], marker='o', color='blue')
70     plt.scatter(pos2[0],pos2[1], marker='o', color = 'yellow')
71     plt.scatter(pos3[0],pos3[1], marker='o', color = 'green')
72
73
74 def update_robot_position(pos1,pos2,pos3, slope_y1, slope_y2, step_size):
75
76     if np.sign(slope_y1)>0 and np.sign(slope_y2)>0 :
77         pos1[1] += step_size
78         pos2[1] += step_size
79         pos3[1] += step_size
80     elif np.sign(slope_y1)<0 and np.sign(slope_y2)<0 :
81         pos1[1] -= step_size
82         pos2[1] -= step_size
83         pos3[1] -= step_size
84
85     return pos1, pos2, pos3
86

```

```

87 # 4. Initialize
88
89 # Initialize The Sinking Vortex Field
90 x = np.arange(x_start, x_end, quiver_step_size)
91 y = np.arange(y_start, y_end, quiver_step_size)
92 X, Y = np.meshgrid(x, y)
93 u, v = spring_stream(X, Y, x_source, y_source)
94
95 # Initialize the Robot's positions
96 pos1=robot_1_start
97 pos2=[(robot_1_start[0] + robot_2_offset[0]),(robot_1_start[1] + robot_2_offset[1])]
98 pos3=[(robot_1_start[0] + robot_3_offset[0]),(robot_1_start[1] + robot_3_offset[1])]
99
100 # Initialize output Plot
101 plt.figure(figsize=(6, 6))
102 plot_robot_position(pos1,pos2,pos3)
103
104
105 # Doing a rotation step prior to the main loop.
106
107 [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
108 [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
109 [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)
110
111 [mag1, angle1] = composite_vector(u1,v1)
112 [mag2, angle2] = composite_vector(u2,v2)
113 [mag3, angle3] = composite_vector(u3,v3)
114
115 angle = (angle1+angle2+angle3)/3 +np.pi/2
116
117 pos1=robot_1_start
118 pos2=[pos1[0] + robot_offset*np.cos(angle),pos1[1] + robot_offset*np.sin(angle)]
119 pos3=[pos2[0] + robot_offset*np.cos(angle),pos2[1] + robot_offset*np.sin(angle)]
120
121
122 plot_robot_position(pos1,pos2,pos3) # Plot Robot Position
123
124
125
126
127 # 5. Main
128
129 for _ in range(1,100):

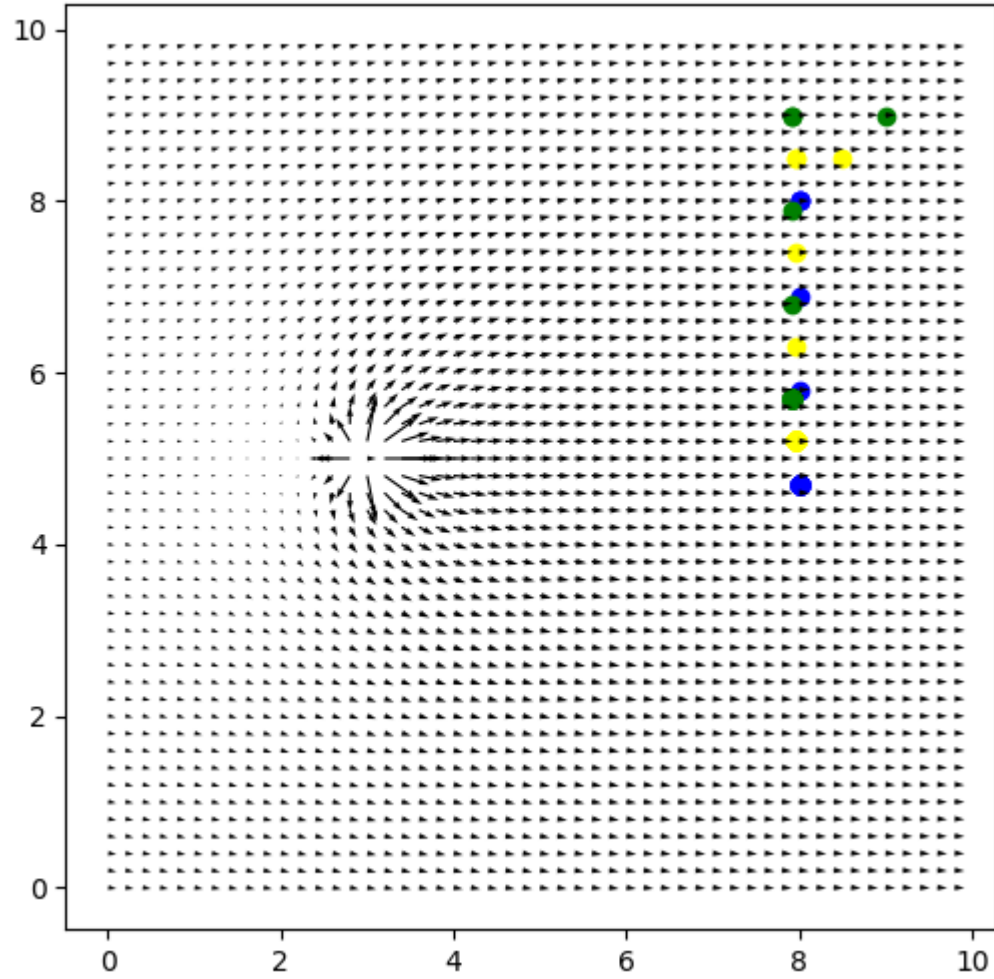
```

```

130     # Check Break Criteria
131
132     # Find u,v, angle and magnitude at each Robot
133
134     [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
135     [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
136     [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)
137
138     [mag1, angle1] = composite_vector(u1,v1)
139     [mag2, angle2] = composite_vector(u2,v2)
140     [mag3, angle3] = composite_vector(u3,v3)
141
142     # Plot Robot Position
143     plot_robot_position(pos1,pos2,pos3)
144
145     # Update Robot Position
146     #angle = -(angle1 + angle2 + angle3)/3
147     #slope_x = (mag2 - mag1) / robot_2_offset[0]
148     slope_y1 = (mag2 - mag1) / robot_2_offset[1]
149     slope_y2 = (mag3 - mag2) / robot_3_offset[1] - robot_2_offset[1]
150
151     #angle = np.arctan2(slope_y,slope_x) #+ np.pi/2
152
153     # Update Robot Position
154
155     [pos1, pos2, pos3] = update_robot_position(pos1,pos2,pos3,slope_y1, slope_y2, robot_step_size)
156
157
158 # 6. Plot Outcome
159
160
161 plt.quiver(x, y, u, v)
162 plt.title(' The Sinking Vortex')
163 #plot_extra_features(x, y, u, v, x_source, y_source)
164 plt.show()
165

```


The Sinking Vortex



Lowest Magnitude Stagnation Point Finder

In [3]:

```
1  ## 1. Import Libraries
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # 2. Define constants
6  x_start = y_start = 0
7  x_end = y_end = 10
8  quiver_step_size = 0.2
9
10 x_source = 3
11 y_source = 5
12
13 robot_1_start = [9,8]
14 robot_2_offset = [0.3,0]
15 robot_3_offset = [0,0.3]
16
17 robot_step_size = 0.5
18 step_size = 1
19
20
21
22 def spring_stream(x, y, x_source, y_source):
23
24     # Establishing a Uniform Flow
25     u = np.ones_like(x)
26     v = np.zeros_like(x)
27
28
29     # Calculating A Source
30     x_centre = x - x_source
31     y_centre = y - y_source
32     r = np.sqrt(x_centre**2 + y_centre**2)
33     r[r==0] = 1e-6
34
35     # Adding the uniform flow to the spring source
36     u = u + x_centre / r**2
37     v = v + y_centre / r**2
38
39     return u, v
40
41 def robot_readings(x, y, x_source, y_source):
42
43     # Centring over sink
```

```

44     x_centre = x - x_source
45     y_centre = y - y_source
46
47     # Calculate the distance r from the center
48     dist = 0.8*np.sqrt((x-x_source)**2 + (y-y_source)**2)
49
50     # Establishing a Uniform Flow
51     u = np.ones_like(x)
52     v = np.zeros_like(x)
53
54     # Adding the uniform flow to the spring source
55     u = u + x_centre / dist**2
56     v = v + y_centre / dist**2
57
58     return u, v
59
60 def composite_vector(u,v):
61
62     mag = np.sqrt(u**2 + v**2)
63     angle = np.arctan2(v,u)
64
65     return mag, angle
66
67 def plot_robot_position(pos1,pos2,pos3):
68
69     plt.scatter(pos1[0],pos1[1], marker='o', color='blue')
70     plt.scatter(pos2[0],pos2[1], marker='o', color = 'yellow')
71     plt.scatter(pos3[0],pos3[1], marker='o', color = 'green')
72
73
74 def update_robot_position(pos1,pos2,pos3, slope_y, slope_x, robot_step_size):
75
76     pos1[0] += step_size*np.cos(angle)
77     pos1[1] += step_size*np.sin(angle)
78     pos2[0] += step_size*np.cos(angle)
79     pos2[1] += step_size*np.sin(angle)
80     pos3[0] += step_size*np.cos(angle)
81     pos3[1] += step_size*np.sin(angle)
82
83     return pos1, pos2, pos3
84
85 # 4. Initialize
86

```

```

87 # Initialize The Sinking Vortex Field
88 x = np.arange(x_start, x_end, quiver_step_size)
89 y = np.arange(y_start, y_end, quiver_step_size)
90 X, Y = np.meshgrid(x, y)
91 u, v = spring_stream(X, Y, x_source, y_source)
92
93 # Initialize the Robot's positions
94 pos1=robot_1_start
95 pos2=[(robot_1_start[0] + robot_2_offset[0]),(robot_1_start[1] + robot_2_offset[1])]
96 pos3=[(robot_1_start[0] + robot_3_offset[0]),(robot_1_start[1] + robot_3_offset[1])]
97
98 # Initialize output Plot
99 plt.figure(figsize=(6, 6))
100 plot_robot_position(pos1,pos2,pos3)
101
102 # 5. Main
103
104 for _ in range(1,11):
105     # Check Break Criteria
106
107     # Find u,v, angle and magnitude at each Robot
108
109     [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
110     [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
111     [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)
112
113     [mag1, angle1] = composite_vector(u1,v1)
114     [mag2, angle2] = composite_vector(u2,v2)
115     [mag3, angle3] = composite_vector(u3,v3)
116
117     # Plot Robot Position
118     plot_robot_position(pos1,pos2,pos3)
119
120
121
122     # Update Robot Position
123     #angle = -(angle1 + angle2 + angle3)/3
124     slope_x = (mag2 - mag1) / robot_2_offset[0]
125     slope_y = (mag3 - mag1) / robot_3_offset[1]
126     angle = np.arctan2(slope_y,slope_x) - np.pi/2
127
128     [pos1, pos2, pos3] = update_robot_position(pos1,pos2,pos3,slope_y, slope_x, robot_step_size)
129

```

```
130     if mag1**2 + mag2**2 + mag3**2 < 3.5 : # and slope_y**2<0.01:
131         break
132
133 # 8. Getting to the stagnation point.
134
135 # 6. Plot Outcome
136
137
138 plt.quiver(x, y, u, v)
139 plt.title(' The Sinking Vortex')
140 #plot_extra_features(x, y, u, v, x_source, y_source)
141 plt.show()
142
```

The Sinking Vortex

